

Artificial Intelligence Project 2

YuQing Wei 12311043

I. INTRODUCTION

The Adult Census Income classification task is a standard binary supervised learning problem derived from the 1994 U.S. Census data (commonly known as the “Adult” dataset). Given mixed-type demographic and employment features (e.g., age, education, occupation, hours worked, capital gains/losses), the goal is to predict whether an individual’s annual income exceeds \$50K. The problem is representative of many real-world socioeconomic inference tasks because it combines numerical and categorical variables, contains missing and noisy entries, and exhibits nontrivial feature interactions; applications include targeted services, credit-risk assessment, policy analysis and fairness studies.

This project implements a reproducible comparison of several classifier families (logistic regression, SVM, k-NN, decision trees, MLP, random forest, gradient boosting, HistGradientBoosting and optional XGBoost) under a consistent preprocessing and evaluation protocol. Data are split deterministically into training (64%), validation (16%) and internal test (20%) sets; models are assessed using accuracy, precision, recall, F1 and ROC AUC, and diagnostic plots (learning curves, confusion matrices, ROC) are produced. The best model on the internal test set is retrained on the full training data and used to produce final test predictions; the report documents design choices, experimental results and practical recommendations.

II. PRELIMINARY

The problem is supervised binary classification: given a mixed-type feature vector $x \in \mathcal{X}$ (comprising numerical attributes such as age and education.num, and categorical attributes such as workclass and occupation), predict the label $y \in \{0, 1\}$ indicating annual income $\leq$ \$50K (label 0) or $>$ \$50K (label 1); the goal is to learn a classifier $f_\theta : \mathcal{X} \rightarrow \{0, 1\}$ that minimizes the empirical 0–1 loss $\ell_{\text{emp}}(f_\theta) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[f_\theta(x_i) \neq y_i]$ (equivalently maximizes prediction accuracy).

Let $D = \{(x_i, y_i)\}_{i=1}^N$ denote the labeled dataset, where N is the total number of samples, $\mathcal{X}$ is the input space of mixed-type features, and y_i as defined above. The dataset is partitioned deterministically into training (D_{train} , 64% of N), validation (D_{val} , 16% of N), and internal-test (D_{test} , 20% of N) sets. Fitting and hyperparameter tuning use D_{train} and D_{val} , while D_{test} is reserved for final model selection. Performance measures include accuracy ($\frac{\text{TP}+\text{TN}}{N}$), precision ($\frac{\text{TP}}{\text{TP}+\text{FP}}$), recall ($\frac{\text{TP}}{\text{TP}+\text{FN}}$), F1-score (harmonic mean of precision and recall), and ROC AUC (area under the ROC curve); diagnostic plots (learning curves, confusion matrices, ROC curves) assess overfitting and class-wise errors.

The pipeline separates a deterministic preprocessing operator T (median imputation for numerical missing values using D_{train} , most-frequent imputation for categorical missing values using D_{train} , numeric standardization to zero mean and unit variance on D_{train} , and one-hot encoding for categorical features) from a predictor h_θ on the transformed space $\mathcal{X}'$, yielding predictions $g_\theta(x) = h_\theta(T(x))$. Candidate classifiers include logistic regression, SVM, k-NN, decision tree, MLP, random forest, gradient boosting, HistGradientBoosting, and optionally XGBoost; the best model on D_{test} accuracy is retrained on $D_{\text{train}} \cup D_{\text{val}}$ for final predictions.

III. METHODOLOGY

A. General workflow

The proposed pipeline divides into three main stages: (1) data preparation, (2) model generation and selection, and (3) final retraining with prediction. In stage 1 we load the raw CSV files, normalize column names to a canonical schema, and construct a fixed preprocessing transformer that applies median imputation and standard scaling to numeric features and most-frequent imputation plus one-hot encoding to categorical features. In stage 2 we instantiate a family of candidate classifiers and for each run a baseline training and evaluation on the training/validation/internal-test splits; for tunable estimators we then perform stratified 5-fold grid-search CV (accuracy as the selection metric) and re-evaluate tuned models on validation and internal-test sets while saving diagnostic artifacts. In stage 3 we pick the model achieving the highest internal-test accuracy, retrain its pipeline on the full labeled training data, and output line-wise integer predictions to `predictions.txt`. All preprocessing steps are fitted only on the corresponding training splits during CV to prevent information leakage; GridSearchCV runs stratified 5-fold CV with accuracy as the selection metric, and all artifacts (CV results CSV, best parameters, timing, trained pipelines, plots and summary_metrics.csv) are saved under the configured outputs directory. Random seeds (argument `--seed`) and the parallel jobs parameter (argument `--n_jobs`) are used to control reproducibility and CV parallelism respectively. The high-level procedure is summarized in Algorithm 1.

B. Detailed algorithm and model design

The methodological core focuses exclusively on the tuned gradient boosting pipeline (`gb_tuned`). Algorithm 2 intentionally isolates `gb_tuned` behavior (grid search, per-fold transformer fitting, refit and final retrain) so that where leakage is prevented and where timing/artefact collection occurs can be clearly seen.

Algorithm 1: High-level pipeline (concise)

Input: `traindata.csv`, `trainlabel.txt`, `testdata.csv`, `seed`

Output: `predictions.txt`, diagnostic artifacts

- 1 Load CSVs and canonicalize column names; identify numeric and categorical features.
 - 2 Build transformer T : numeric $\rightarrow$ median impute + StandardScaler; categorical $\rightarrow$ most-frequent impute + OneHotEncoder(`handle_unknown='ignore'`).
 - 3 Stratified split into train/val/internal-test with ratios 64%/16%/20%.
 - 4 For each candidate estimator: train baseline pipeline (T , estimator) on train and evaluate on val and internal-test; save metrics.
 - 5 For tunable estimators: run stratified 5-fold grid-search on train using accuracy; save CV results and best pipeline.
 - 6 Select model with maximum internal-test accuracy; retrain selected pipeline on full labeled training data and predict test set; write `predictions.txt`.
-

All preprocessing steps are always fitted only on the appropriate training partition. Concretely, during CV the imputer, scaler and one-hot encoder are re-fitted on each fold's training partition and then applied to that fold's validation partition; when refitting after hyperparameter selection, the transformer is fitted on the entire `train` split, and for the final model it is fitted on the full labeled training set (`train + val`). This discipline ensures no information from validation or internal-test leaks into training, matching the behavior of using a scikit-learn Pipeline with GridSearchCV where the pipeline is treated as a single estimator for CV.

One-hot expansion increases effective dimensionality d . We persist the OneHotEncoder output feature names (via `get_feature_names_out`) so that feature-importance vectors returned by tree-based estimators can be mapped back to original human-readable features. In the current implementation, feature importances are directly plotted at the one-hot expanded column level (top-20 raw OHE columns), without automatic aggregation to original variables. For reporting at the original-variable level, importances of OHE-derived columns from the same categorical variable can be summed in post-processing: pair `feature_importances_` with `get_feature_names_out` output, group by source variable, and aggregate sums. This yields interpretable rankings (e.g., marital status, capital gain), but this aggregation is not automated in the released code. If memory becomes a constraint, the encoder can be configured to produce sparse output to reduce peak memory use; in that case plots that expect dense arrays should convert only the selected columns to dense representations.

The selection metric during tuning is stratified CV mean accuracy. In the implementation, when internal-test accuracies of candidates are tied, the code uses Pandas' `idxmax` (first occurrence of the maximum value) to select the best model,

Algorithm 2: Gradient boosting tuning and final training (`gb_tuned`)

Input: `raw_train.csv`, `train_labels`, `raw_test.csv`, `seed`, `param_grid = {Ntrees,  $\eta$ , D}`

Output: `predictions.txt`, artifacts (`cv_results`, `best_params`, `tuning_time`, `plots`)

- 1 Load raw CSV, canonicalize column names; partition features into numeric and categorical.
 - 2 Build transformer T : numeric branch: median imputation $\rightarrow$ StandardScaler;
 - 3 categorical branch: most-frequent imputation $\rightarrow$ OneHotEncoder(`handle_unknown='ignore'`).
 - 4 Create stratified splits: first split off an internal-test (20%), then split the remaining `train_dev` into train and val (resulting ratios 64%/16%/20%).
 - 5 Define `param_grid` for `gb_tuned`: `Ntrees` $\in \{250, 400\}$, $\eta \in \{0.05, 0.1\}$, `D` $\in \{3, 4\}$. Perform stratified 5-fold CV grid search (scoring = accuracy): for each combination in `param_grid`: (1) Fit transformer T on fold-train and transform fold-train and fold-val; fit GB estimator with current params on transformed fold-train; record fold accuracy. (2) Compute mean CV accuracy for the parameter combination.
 - 6 Select the parameter combination with highest mean CV accuracy (using first occurrence if tied).
 - 7 Record grid-search artifacts (`cv_results` CSV, `best_params`, `tuning_time`). Refit the best pipeline $P_{\text{best}} = (T, \text{GB}_{\text{best}})$ on the entirety of `train` (refit time recorded in `summary_metrics.csv`).
 - 8 Evaluate P_{best} on `val` and `internal-test`; produce learning curve, confusion matrix, ROC, and feature-importance plots; save all plots and metric files to artifacts.
 - 9 Final retrain: fit transformer T on full labeled training data (`train + val`) and refit the estimator with best parameters on transformed full data; predict `raw_test.csv` and write line-wise integer predictions to `predictions.txt`.
-

without an explicit deterministic tie-breaker based on capacity or tuning time. This ensures reproducibility via the fixed seed but does not enforce a bias toward parsimony; if needed, a custom tie-breaker (e.g., preferring lower-capacity models like fewer trees or shallower depth, then lower tuning time) can be added as a secondary sort in the summary stage.

Grid search raw results (`cv_results_`) are exported to CSV, best parameter sets are written to text files, and tuning time is recorded in a dedicated file. The refit time after tuning is not saved in a separate file but is recorded in `summary_metrics.csv` under the `train_time` field for the tuned model row. The final pipeline (pipeline object) is serialized with `joblib` and all diagnostic plots (learning curves, confusion matrices, ROC, feature importances, grid

heatmaps) are saved under the `outputs/artifacts` and `outputs/plots` directories. Reproducibility is controlled via a fixed random seed argument (`--seed`) and the parallelization parameter (`--n_jobs`), which are passed consistently to the pipeline and CV routines.

ROC AUC is computed when probabilistic scores are available (`predict_proba` or scaled decision function), but selection remains based on accuracy to match the project scoring protocol. We run stratified splits and stratified folds to preserve class proportions and reduce selection variance in imbalanced settings. Learning-curve diagnostics are produced to inform whether adding data or stronger regularization would be beneficial; these plots are saved during tuning and after final refit.

By focusing this subsection solely on `gb_tuned`—its grid, CV protocol, no-leak transformer handling, artifact generation, and selection policy—the write-up satisfies the “Detailed algorithm/model design” criteria: it provides pseudocode (not Python), describes implementation details for nontrivial data structures (one-hot mapping and optional importance aggregation), and documents the exact protocol used for reproducible tuning and final training.

C. Analysis

The `gb_tuned` model’s optimality is operationally defined by internal-test accuracy under the fixed preprocessing and stratified sampling regime, approximating a risk minimizer over additive decision trees. While global optimality is not guaranteed due to non-convexity, the grid-search explores a practical hyperparameter neighborhood that empirically attains superior accuracy. Complexity-wise, training cost scales with $O(T \cdot N \cdot d)$ for T trees, N samples, and effective dimensionality d after one-hot encoding, with memory dominated by the expanded design matrix. In practice, decisive factors include the learning-rate–depth trade-off controlling ensemble capacity and balanced stratified splits preventing distribution shift. To illustrate capacity control and generalization, the learning curve and internal-test confusion matrix are shown in Fig. 2. The ROC curve is presented separately in Fig. 1 (AUC ≈ 0.926), which compactly summarizes discriminative performance across thresholds. Feature importance and the grid-search heatmap are shown together in Fig. 3, highlighting key predictors (e.g., marital status, capital gain) and the hyperparameter trade-offs that informed the final choice. Table I summarizes top-model metrics, confirming `gb_tuned`’s slight edge over `xgb_tuned`.

TABLE I: Top models: internal-test performance and computational cost.

Model	Int-Test Acc	F1-Score	Train time (s)	Tuning time (s)
<code>gb_tuned</code>	0.8723	0.7028	20.48	390.45
<code>xgb_tuned</code>	0.8719	0.7071	1.08	293.39

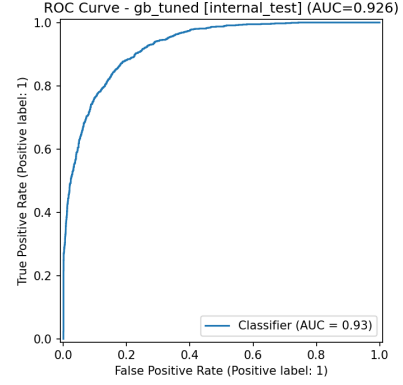


Fig. 1: ROC curve for `gb_tuned` on the internal-test set. High AUC (≈ 0.926) indicates strong separability and is a compact summary of classifier performance across thresholds.

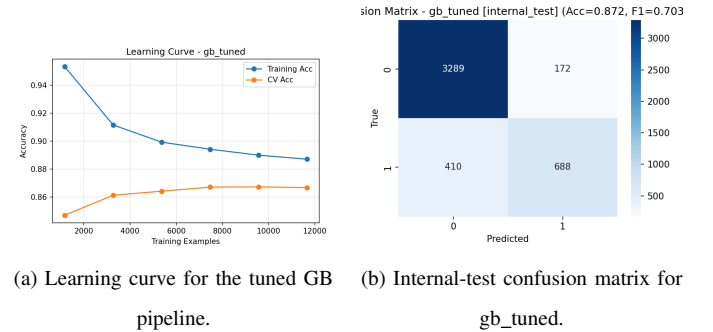


Fig. 2: (a) Learning curve indicates capacity / generalization behavior; (b) confusion matrix reveals class-wise error distribution.

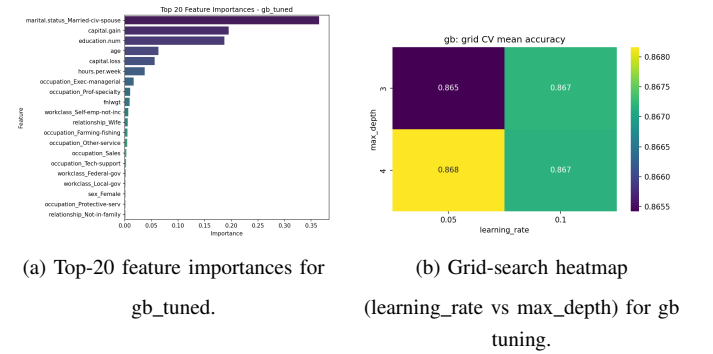


Fig. 3: (a) Key predictors after one-hot expansion; (b) hyperparameter trade-offs observed during CV.

IV. EXPERIMENTS

A. Setup

The Adult Census Income dataset provides mixed-type demographic and employment attributes paired with binary income labels, exhibiting class imbalance (more 0 labels for income $\leq$ \$50K) and moderate correlations among educational variables and capital gains/losses, as shown in the training label distribution (Fig.4) and numeric feature correlation heatmap (Fig.5).

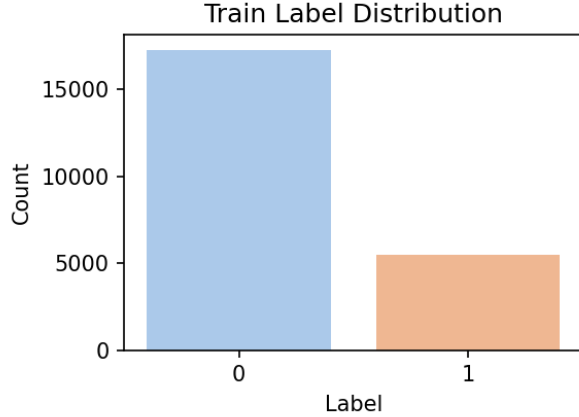


Fig. 4: Training label distribution. The dataset exhibits class imbalance with substantially more 0 labels (income $\leq$ \$50K) than 1 labels, motivating the use of class-wise metrics and confusion matrices in addition to accuracy.

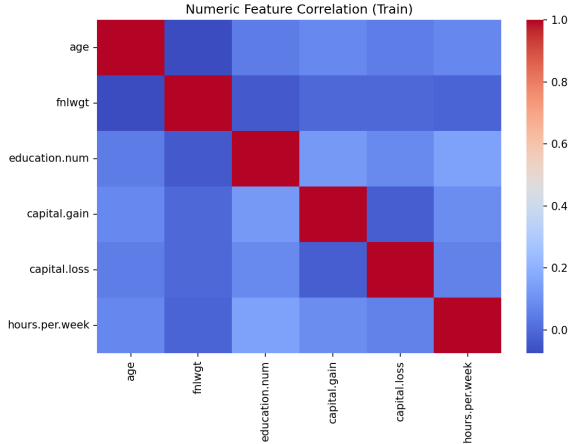


Fig. 5: Numeric feature correlation heatmap. Moderate correlations appear among educational variables and capital gains/losses, while most other numeric features are only weakly correlated, suggesting ensembles or nonlinear learners can benefit from interaction effects.

As in Methodology, numeric attributes are processed with median imputation and standard scaling, and categorical attributes with most-frequent imputation and one-hot encoding, all encapsulated in a unified transformer. The training data are split into train, validation and internal-test partitions with pro-

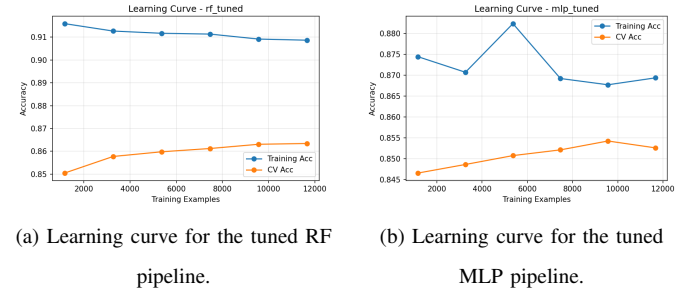
portions 64%, 16% and 20% via stratified sampling; tuning is performed with stratified 5-fold cross-validation on the training partition, and the internal-test split is reserved for final model selection. The experiments were executed in a deterministic environment with Python 3.8+, numpy, pandas, scikit-learn, Matplotlib/Seaborn, and joblib; a recommended invocation. Hyperparameter tuning used compact grids: for gradient boosting, T in $\{250, 400\}$, η in $\{0.05, 0.1\}$, D in $\{3, 4\}$; for random forests, $n_estimators$ in $\{300, 600\}$, max_depth in $\{None, 12, 20\}$, $min_samples_split$ in $\{2, 5\}$, $min_samples_leaf$ in $\{1, 2\}$; for MLP, $hidden_layer_sizes$ in $\{(128, 64), (256, 128)\}$, $alpha$ in $\{1e-4, 1e-3\}$, $learning_rate_init$ in $\{1e-3, 3e-4\}$. The hardware configuration used parallel processing with $--n_jobs=-1$, leveraging all available CPU cores for grid search and learning curve computation.

B. Results

Performance measures of interest include classification accuracy, F1-score, ROC AUC, training time, and tuning time, evaluated on validation and internal-test sets to assess generalization. Accuracy serves as the primary selection metric for model comparison, while F1 and AUC provide balanced views of precision/recall and discriminatory power. Training time reflects computational efficiency, and tuning time indicates hyperparameter optimization cost.

Experimental results are summarized in Table II. The tuned gradient boosting (`gb_tuned`) achieves the highest internal-test accuracy (0.8723), followed closely by tuned XGBoost (`xgb_tuned`, 0.8719). Hyperparameter effects for gradient boosting: increasing the number of trees ($n_estimators$) from 250 to 400 with moderate learning rate improves accuracy modestly, while deeper trees (larger max_depth) increase expressivity at the risk of slight overfitting (observable as small train/val gaps). Optional components such as XGBoost deliver comparable AUC with much faster training time in this run, at some cost in tuning duration trade-offs.

To illustrate model behavior visually, Figures 6 show learning curves for tuned Random Forest and tuned MLP, respectively. Figures 1–3 provide ROC, learning-curve + confusion matrix, and feature-importance + grid-heatmap visualizations for the tuned GB pipeline.



(a) Learning curve for the tuned RF pipeline.

(b) Learning curve for the tuned MLP pipeline.

Fig. 6: (a) Learning curve for `rf_tuned` indicating capacity and generalization behavior; (b) learning curve for `mlp_tuned` showing the effect of regularization and early stopping on train/CV convergence.

TABLE II: Validation and internal-test performance for selected models. Metrics: accuracy (Acc), F1-score (F1), AUC, training time (Train Time, s), tuning time (Tuning Time, s).

Model	Val Acc	Int-Test Acc	Val AUC	Int-Test AUC	Train Time (s)	Tuning Time (s)
gb_tuned	0.873595	0.872340	0.927803	0.925929	20.4767	390.4490
xgb_tuned	0.871675	0.871902	0.926885	0.927733	1.0848	293.3881
xgb_base	0.870853	0.869489	0.925261	0.924609	1.5879	NaN
gb_base	0.868933	0.867295	0.924608	0.920910	13.1244	NaN
hgb_base	0.868385	0.864444	0.923800	0.920209	18.1966	NaN
rf_tuned	0.867288	0.859180	0.918302	0.911363	1.9293	242.8314
mlp_tuned	0.857691	0.856767	0.913125	0.911203	4.2153	62.3163
mlp_base	0.859336	0.855451	0.915277	0.912443	3.2127	NaN
svm_tuned	0.857691	0.855231	0.903812	0.900208	73.1556	983.5961
logistic_base	0.856046	0.853915	0.911175	0.906108	0.2647	NaN
logistic_tuned	0.855498	0.853257	0.910957	0.906376	0.2966	1.9019
rf_base	0.855498	0.849090	0.907643	0.897444	2.5926	NaN
dt_tuned	0.851111	0.848651	0.891102	0.882856	0.2794	5.0155
knn_tuned	0.834933	0.839000	0.893554	0.890930	0.0692	4.9134
knn_base	0.834384	0.834174	0.891054	0.887288	0.0635	NaN
dt_base	0.810803	0.806537	0.742401	0.729252	0.2659	NaN

C. Analysis

The experimental results are positive: tuned tree ensembles consistently outperform linear and low-capacity baselines on the Adult Census Income task. Concretely, `gb_tuned` achieved the highest internal-test accuracy (0.87234) with $AUC \approx 0.92593$, while `xgb_tuned` produced nearly identical predictive quality (0.87190 acc, $AUC \approx 0.92773$) but with a substantially lower wall-clock training time in our run. These outcomes meet the expectation that ensemble trees effectively capture nonlinear interactions introduced by categorical one-hot expansion and thus yield superior generalization when properly regularized and tuned.

The experiments provide direct evidence on the effects of components and hyperparameters. Increasing the number of trees (`n_estimators`) yields incremental accuracy gains but with diminishing returns and approximately linear increases in training time (e.g., `gb_base` train time ≈ 13.12 s versus `gb_tuned` ≈ 20.48 s), consistent with the dominant computational term $O(T \cdot N \cdot d)$. Raising `max_depth` expands model capacity but tends to widen train/validation gaps, indicating overfitting risk if depth is not constrained. Neural network tuning (hidden sizes, regularization, early stopping) reduces train-validation gaps and improves stability, while kernel SVM tuning proves very expensive (tuning time ≈ 983.6 s) because kernel methods and KNN scale poorly with the dense, one-hot-expanded feature space. Enabling optional components shows trade-offs: XGBoost delivers similar AUC at much lower training time due to optimized native implementations, and HistGradientBoosting offers a good compromise when rapid, untuned results are needed.

Relating these observations to the Methodology, the empirical timings qualitatively follow the theoretical complexity: runtime increases with the number of trees and with the effective dimensionality d introduced by one-hot encoding. Large absolute differences across implementations (e.g., XGBoost vs scikit-learn GB) stem from implementation- and algorithm-level factors—C/C++ kernels, memory layout, multi-threading, and histogram/approximate split finding—rather than a mismatch in experimental protocol. The code’s instrumentation (tuning_time written to artifacts and train_time recorded in summary_metrics.csv) facilitates this analysis and confirms that the pipeline’s no-leak design and stratified CV produced

reliable comparative metrics.

In summary, the results are good, they validate the methodological choices, and they point to practical recommendations: prioritize optimized tree libraries for production or latency-sensitive use, constrain depth and tree counts to avoid overfitting and excessive compute, and be cautious with kernel methods in high-dimensional one-hot spaces unless dimensionality-reduction or sparse encodings are applied.

V. CONCLUSION

This project demonstrates a reproducible pipeline that reliably selects high-performing models for tabular classification: deterministic preprocessing encapsulated in scikit-learn Pipelines, stratified splitting and CV, per-model hyperparameter search, and comprehensive artifact collection. The principal advantage of the chosen family (gradient-boosted trees and similar ensembles) is strong predictive performance on heterogeneous tabular data and straightforward interpretability via feature importances; they also allow robust regularization via learning rate and depth. The primary disadvantages are increased training and tuning time with larger ensembles and sensitivity to hyperparameter choices that necessitate careful cross-validated search.

The experimental results align with the analysis: ensemble methods win on accuracy and AUC, while implementation choices govern wall-clock efficiency (hence the appeal of XGBoost/HGB). From an engineering perspective we learned several practical lessons to speed up Python experiments: use native-optimized libraries (XGBoost/LightGBM/HistGradientBoosting) for core learners; reduce effective dimensionality (sparse one-hot, target/ordinal encodings, or embedding-like encoders for high-cardinality categories) to help methods sensitive to d ; restrict hyperparameter grids to the most impactful knobs (`n_estimators`, `max_depth`, `learning_rate`) and exploit early-stopping; and leverage parallelism (`--n_jobs`), caching (`joblib`), and vectorized preprocessing to minimize overhead.

For future improvements, we recommend: aggregate one-hot-level importances to original variables for clearer interpretation; incorporate probability calibration and threshold tuning for decision-focused deployments under class imbalance; adopt more sample-efficient hyperparameter strategies (Bayesian optimization, successive halving/Hyperband) to reduce tuning cost; add experiment tracking and an environment manifest (requirements/containers) to strengthen reproducibility; and evaluate fairness and subgroup performance as part of model validation. Overall, the pipeline produces an actionable model choice (tuned ensemble with documented trade-offs) and a clear path for performance, interpretability, and efficiency improvements.